

Managing cloud based calibre libraries

Rohit Goswami · 2023-08-19 · workflow · tools · ramblings · books

Ruminations on cross-cloud library management

Background

I have a set of different devices across which I need to sync my library (described in [an earlier post](#)). Ideally this should be a lazy synchronization mechanism and be as general as possible.

Constraints

`calibre` should never be modifying the files while they are being synced, not only because it creates and removes a lot of temporary files and folders which will cause churn, but also because it actually locks files during modification and this leads to permission issues.

Approximations to solutions

0th: Cloud native synchronization tools {#0th-cloud-native-synchronization-tools}

I use `pCloud`, which has a nifty GUI client (and CLI) across my operating systems which essentially provides a mounted drive.

- Store directly on the drive
 - Very slow, since many directories and temporaries are created

1st: GUI based synchronization {#1st-gui-based-synchronization}

Next we can consider using `sync` via the GUI. This essentially binds two folders together and continuously ensures they are up to date.

- This syncs continuously and so ended up with permission errors

2nd: `rsync` to mounted drive {#2nd-rsync-to-mounted-drive}

As a slightly more complicated workflow we can use a local synchronization tool on a `cron` schedule to sync to the drive mount.

```
rsync -avP /path/to/source/directory/ /path/to/destination/directory
```

Pros

- `cron` job can be linked to triggers like:
 - Checking if `calibre` is running
 - Checking for network / power requirements

Cons

- This doesn't really sync immediately, and the delay might be pretty significant if earlier jobs are skipped

3rd: Use `rclone` manually {#3rd-use-rclone-manually}

`rsync` is really meant for SSH connections, while `Rclone` is fantastic for working with cloud providers. Setting this up is a breeze.

```
rclone config
# Best to encrypt it
rclone sync -P /path/to/calibre/library remote-name:/path/in/pcloud
```

Pros

- Simple setup

Cons

- Manual, so this will need to be run every time

Coupling to `calibre`

`calibre` is the only program which is supposed to manage the libraries. In that sense, it makes sense to strongly couple syncing with `calibre`. Most of my machines have `gnome-keyring` installed, and `rclone` supports secrets stored in password managers so:

```
secret-tool store --label="rclone" service "rclone" user "calisync"
```

Which can then simply be set to `RCLONE_PASSWORD_COMMAND` and used directly:

```
export RCLONE_PASSWORD_COMMAND='secret-tool lookup service "rclone" user "calisync"'
rclone sync -P CalibreLibs/Synced pCloud:/Calibre --ask-password=false
```

Putting this together, we can wrap `calibre` runs in a script:

```
#!/bin/bash
# $HOME/.local/bin/calibre_and_sync.sh

# Start calibre and wait
calibre
wait $! # optional really, but a bit safer

# Setup and run sync
RCLONE_PASSWORD_COMMAND='secret-tool lookup service "rclone" user "calisync"'
rclone sync -P ~/CalibreLibs/Synced pCloud:/Calibre --ask-password=false
```

It can be made more fancy if necessary, but should ideally just be customized for each machine. In any case the script is mostly to be used from a `.desktop` file, e.g. in `.local/share/applications/Calisync`.

```
[Desktop Entry]
Type=Application
Name=CaliSync
Exec=/home/username/.local/bin/calibre_and_sync.sh
Icon=calibre-gui
GenericName=E-book library management with cloud sync
Comment=Start Calibre and sync library with pCloud
Terminal=false
Categories=Office;Utility;
```

Do keep in mind that environment variables are not expanded, so generalizing this would require a suitable dotfile management system like `chezmoi` (as [seen here](#)).

Visual Tweaks

The script discussed in the previous section is fine, but it is often nicer to have a visual indication of what's going on in terms of the upload, especially if there are a lot of files.

```
#!/bin/bash
# $HOME/.local/bin/calibre_and_sync.sh

# Start Calibre and wait
calibre

# Setup and run sync
zenity --notification --text="Start syncing"
RCLONE_PASSWORD_COMMAND='secret-tool lookup service "rclone" user "calisync"'
(
    rclone sync -P ~/CalibreLibs/Synced pCloud:/Calibre --ask-password=false \
        --password-command="$RCLONE_PASSWORD_COMMAND" 2>&1 | \
    while read -r line; do
        echo "# $line"
    done
) |
zenity --progress \
    --title="Syncing Calibre Library with pCloud" \
    --text="Syncing..." \
    --percentage=0 \
    --pulsate \
    --auto-close \
    --auto-kill
zenity --notification --text="Sync completed"
```

This is nice, but doesn't actually show the full progress. `zenity` doesn't support embedding a terminal, so the only other option is to spawn a shell.

```
#!/bin/bash
# $HOME/.local/bin/calibre_and_sync.sh

# Start Calibre and wait
calibre

# Setup and run sync
zenity --notification --text="Start syncing"
RCLONE_PASSWORD_COMMAND='secret-tool lookup service "rclone" user "calisync"'
foot -- sh -c "rclone sync -P ~/CalibreLibs/Synced pCloud:/Calibre --ask-password=false --password-command=\"\$RCLONE_PASSWORD_COMMAND\""
zenity --notification --text="Sync completed"
```

Integrating with `zotero`

I've discussed working with [Zotero before](#), but when working with cloud synced calibre libraries, to prevent duplication of files, relative linking of base directory attachments works really well^[1]. [TurtleTech](#) has a [solid writeup](#) on the full WebDAV sync setup, which also covers migrating from Mendeley.

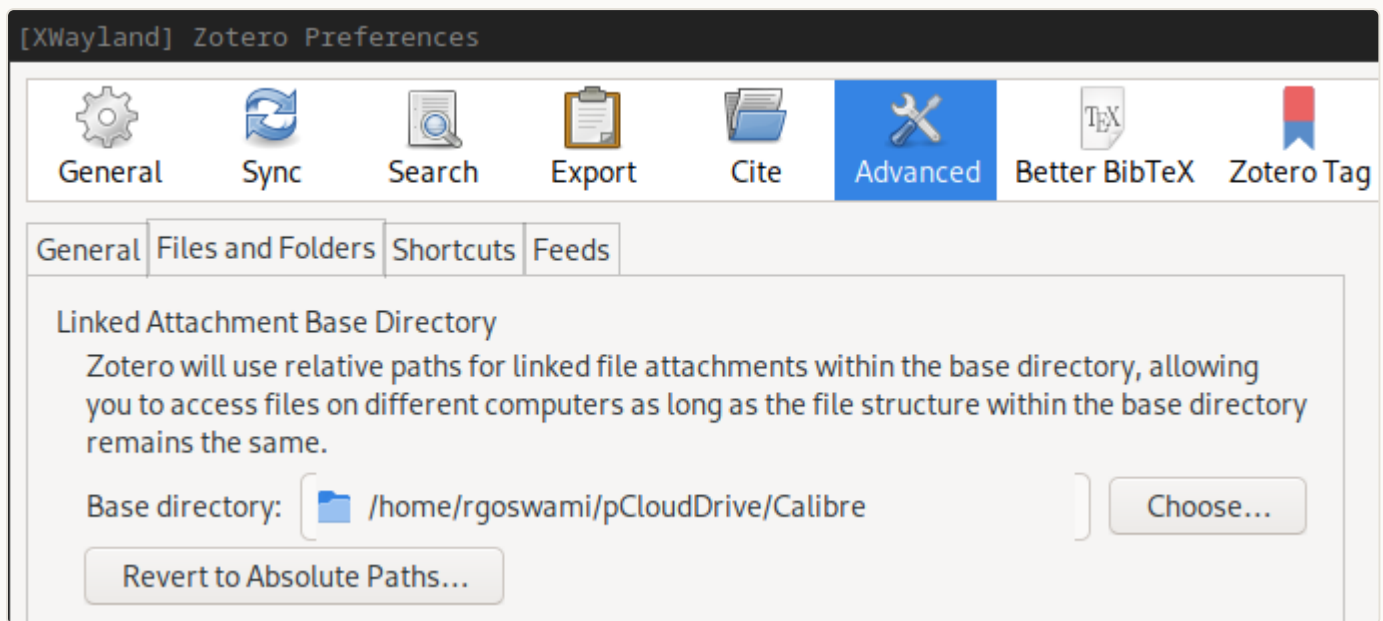


Figure 1: Zotero settings for linking files consistently

Some reasons to do this:

- The main goal of this is to enable keeping notes per-book, which Calibre doesn't support
 - Also to get easy access without duplicated files on the backing WebDAV service (pCloud in my case, or via [TurtleTech](#))
- Allows easier, more natural `bibtex` handling in Zotero, without going through a sync plugin like [ZMI](#)

However:

- Do **not** rename from Zotero, this will break integration with Calibre

Conclusions

With this setup, it is now fairly trivial to sync libraries across devices, and thus, to take advantage of suitably large storage devices and local `calibre` readers. Android devices and `calibre` library support has been a bit of a roller-coaster, especially since Calibre Companion was [sold off and abandoned](#), but for now, it works, as does [Calibre Sync](#) and [KoReader](#). That being said, the best approach would be pairing this setup with something like [FolderSync](#) and [Leger Calibre](#) would be the best but Leger Calibre has gone the [way of the dodo](#), leaving one with precious little options, though [KoReader](#) is still the best option for local `calibre` management without cover information.

`reprex` addendum

This post was executed with binaries at the following versions:

```
rcclone version
rcclone v1.63.1
# pcloud 1.14.0
rsync --version
rsync version 3.2.7 protocol version 31
Copyright (C) 1996-2022 by Andrew Tridgell, Wayne Davison, and others.
zenity --version
3.44.2
```